

BOOK 4 — EXCEPTION HANDLING, MULTITHREADING & SYNCHRONIZATION

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — EXCEPTION HANDLING

1.1 Exception Definition

An **Exception** is an unwanted or unexpected event that occurs during the execution of a program (at runtime) and disrupts the normal flow of instructions.

- **Definition:** An object representing an abnormal condition or runtime error condition.
- **Key Characteristics:** Unlike compile-time syntax errors, exceptions occur dynamically during execution when invalid operations are attempted (e.g., dividing by zero, accessing an out-of-bound array index, opening a non-existent file).

1.2 Need for Exception Handling

Without exception handling, when an error occurs at runtime, Java immediately terminates program execution abnormally, skipping all subsequent code.

- **Goal:** To convert abnormal program termination into a graceful, controlled response and ensure application robustness.
- **Real-World Analogy:** A bank ATM machine encountering a network failure. Instead of crashing completely and locking the user's card inside, it catches the error, displays an informative message ("Transaction failed due to network issue"), and cleanly ejects the card.

1.3 Default Exception Handler

When an exception occurs inside a method:

1. The method creates an **Exception Object** containing the exception name, description, and execution state (stack trace).
2. The method hands over this exception object to the Java Runtime Environment (JRE) — a process called **throwing an exception**.
3. The JRE searches the call stack for a matching `catch` block to handle it.

4. If no user-defined handler is found, the JRE hands the exception to the **Default Exception Handler** (part of the JVM).
5. The Default Exception Handler prints the **Exception Name**, **Description**, and **Stack Trace** to the console and **abnormally terminates** the program.

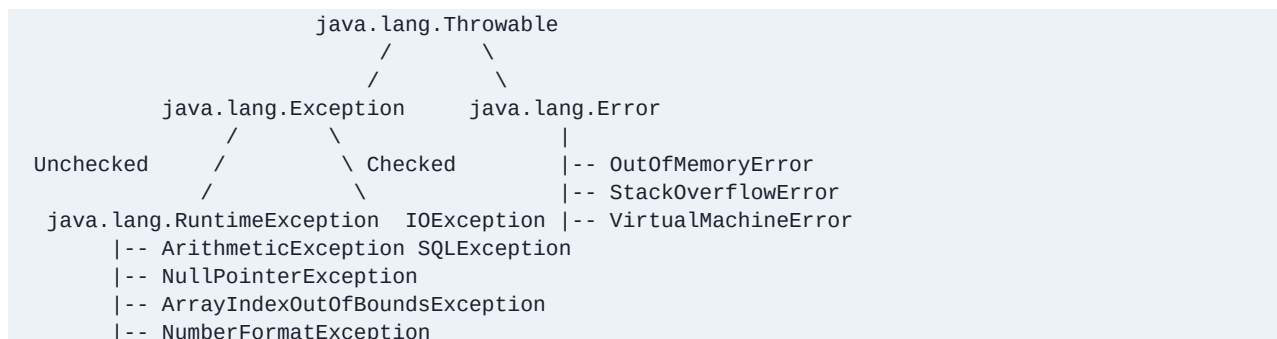
1.4 Exception Propagation

Exception Propagation refers to the mechanism where an unhandled exception is automatically dropped down the call stack from the current method to the calling method.

- **Mechanism:** If method `C()` throws an exception and does not handle it, `C()` pops off the stack frame, and the exception is propagated to caller `B()`. If `B()` does not handle it, it propagates to `A()`, and finally to `main()`. If `main()` leaves it unhandled, the Default Exception Handler takes over and terminates the application.

1.5 Exception Hierarchy

All exception and error types in Java inherit from the root superclass `java.lang.Throwable`.



- **`Throwable`**: The root class of the Java exception hierarchy.
- **`Error`**: Represents serious problems that a reasonable application should not try to catch (e.g., `OutOfMemoryError`, `StackOverflowError`). Errors are caused by environment/system failures.
- **`Exception`**: Represents conditions that a reasonable application might want to catch and recover from.

1.6 `try` Block

The `try` block contains statements that might throw an exception during execution.

- **Rule:** A `try` block cannot exist alone; it must be immediately followed by at least one `catch` block or a `finally` block.

1.7 `catch` Block

The `catch` block contains the exception handling code that executes only when a matching exception is thrown inside its corresponding `try` block.

- **Syntax:** `catch (ExceptionType e) { // Handling logic }`

1.8 `finally` Block

The `finally` block contains cleanup code (e.g., closing database connections, closing files) that is **guaranteed to execute**, regardless of whether an exception is thrown or caught.

- **Execution Rule:** `finally` executes even if `try` or `catch` contains a `return` statement.
- **Only Exception:** `finally` will NOT execute if `System.exit(0)` is explicitly called, or if the JVM experiences a fatal crash.

1.9 Multiple `catch` Blocks

A single `try` block can be followed by multiple `catch` blocks to handle different exception types specifically.

- **Critical Ordering Rule:** Catch blocks must be ordered from **most specific (subclass) to most general (superclass)**. Placing a parent class like `Exception` before a child class like `ArithmeticException` causes a **compilation error** ("exception has already been caught").

1.10 `throw` Keyword

The `throw` keyword is used to explicitly throw an exception object from any method or block of code.

- **Syntax:** `throw new SpecificException("Error Message");`
- **Use Case:** Throwing custom or predefined exceptions based on business logic validations (e.g., validating age for voting eligibility).

1.11 `throws` Keyword

The `throws` keyword is used in a method signature to declare that the method may throw specific exceptions to its caller.

- **Purpose:** Delegates the responsibility of handling checked exceptions to the calling method.
- **Syntax:** `void readFile() throws IOException, SQLException { ... }`

1.12 Checked Exceptions (Compile-Time Exceptions)

Exceptions that are checked by the compiler at compile-time.

- **Requirement:** The compiler forces the developer to handle checked exceptions using either a `try-catch` block or declare them using `throws`.
- **Examples:** `IOException`, `SQLException`, `ClassNotFoundException`, `FileNotFoundException`.
- **Note:** Checked exceptions do NOT inherit from `RuntimeException`.

1.13 Unchecked Exceptions (Runtime Exceptions)

Exceptions that occur at runtime and are NOT checked by the compiler.

- **Requirement:** Handling is optional; the program compiles cleanly even without `try-catch` or `throws`.

- **Examples:** `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`.
- **Hierarchy:** All unchecked exceptions inherit directly or indirectly from `java.lang.RuntimeException`.

1.14 Custom Exceptions (User-Defined Exceptions)

Developers can create custom application-specific exceptions by extending existing Java exception classes.

- **Custom Unchecked Exception:** Extend `java.lang.RuntimeException`.
- **Custom Checked Exception:** Extend `java.lang.Exception`.

1.15 Exception Handling During Method Overriding

When overriding a superclass method in a subclass:

1. **If Superclass Method Declares NO Checked Exception:** The subclass overriding method **cannot declare any new or broader checked exception**. It can, however, declare unchecked exceptions or narrower exceptions.
2. **If Superclass Method Declares a Checked Exception:** The subclass overriding method can declare the **same checked exception**, a **subclass (narrower) checked exception**, or **no exception at all**, but CANNOT declare a broader or new checked exception.
3. **Unchecked Exceptions:** Have no restrictions during method overriding.

1.16 `StackOverflowError`

A fatal error that occurs when a program exhausts the allocated Stack memory.

- **Common Cause:** Infinite recursion (recursive calls without a valid base case or reaching maximum stack depth).
- **Type:** It is an `Error` (inherits from `java.lang.Error`), not an `Exception`.

1.17 Error vs Exception Comparison

Feature	Exception	Error
Root Class	<code>java.lang.Exception</code>	<code>java.lang.Error</code>
Cause	Program logic or user input flaws	System resource exhaustion / JVM failure
Recoverability	Recoverable via <code>try-catch</code>	Unrecoverable; application should crash
Type	Checked or Unchecked	Always Unchecked
Examples	<code>NullPointerException</code> , <code>IOException</code>	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>

1.18 Common Java Exceptions

1. `ArithmeticException`: Arises from invalid arithmetic operations, such as integer division by zero (`10 / 0`).

2. **NullPointerException (NPE)**: Occurs when attempting to invoke a method or access a field on a reference variable that holds `null`.
3. **ArrayIndexOutOfBoundsException**: Triggered when attempting to access an array index that is negative or greater than or equal to `array.length`.
4. **NumberFormatException**: Thrown when attempting to parse a string into a numeric value (e.g., `Integer.parseInt("abc")`), but the string lacks an appropriate numeric format.
5. **ClassCastException**: Thrown when attempting to downcast an object to a subclass type of which it is not an instance.

1.19 try-with-resources (Java 7 Feature)

A specialized `try` block that automatically closes resources (like files, streams, or database connections) when execution completes.

- **Requirement**: The resource object must implement the `java.lang.AutoCloseable` or `java.io.Closeable` interface.
 - **Advantage**: Eliminates explicit resource cleanup code in `finally` blocks, preventing resource leaks.
-

1.20 Common Exception Handling Programs (Syllabus 18.20.1 to 18.20.13)

Problem 1: ArithmeticException Handling (18.20.1)

```
// Demonstration of catching integer division by zero
public class ArithmeticExceptionDemo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        try {
            // Attempting integer division by zero throws ArithmeticException
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            // Gracefully catching and handling the exception
            System.out.println("Handled Exception: Cannot divide an integer by zero!");
            System.out.println("Exception Message: " + e.getMessage());
        }
        System.out.println("Program execution continues normally...");
    }
}
/*
Expected Output:
Handled Exception: Cannot divide an integer by zero!
Exception Message: / by zero
Program execution continues normally...
*/
```

Problem 2: ArrayIndexOutOfBoundsException Handling (18.20.2)

```
// Demonstration of catching invalid array index access
public class ArrayIndexDemo {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};

        try {
            // Array length is 3 (valid indices: 0, 1, 2). Accessing index 5 throws exception.
            System.out.println("Accessing index 5: " + numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Handled Exception: Attempted to access an invalid array index!");
            System.out.println("Invalid Index Details: " + e.getMessage());
        }
    }
}
/*
Expected Output:
Handled Exception: Attempted to access an invalid array index!
Invalid Index Details: Index 5 out of bounds for length 3
*/
```

Problem 3: `NullPointerException` Handling (18.20.3)

```
// Demonstration of handling method call on null reference
public class NullPointerExceptionDemo {
    public static void main(String[] args) {
        String str = null;

        try {
            // Invoking a method on null throws NullPointerException
            int length = str.length();
            System.out.println("Length: " + length);
        } catch (NullPointerException e) {
            System.out.println("Handled Exception: Cannot invoke methods on a null reference!");
        }
    }
}
/*
Expected Output:
Handled Exception: Cannot invoke methods on a null reference!
*/
```

Problem 4: `NumberFormatException` Handling (18.20.4)

```
// Demonstration of handling invalid string-to-integer conversion
public class NumberFormatDemo {
    public static void main(String[] args) {
        String invalidNumberStr = "123a";

        try {
            // Parsing non-numeric string throws NumberFormatException
            int parsedValue = Integer.parseInt(invalidNumberStr);
            System.out.println("Parsed Value: " + parsedValue);
        } catch (NumberFormatException e) {
            System.out.println("Handled Exception: Failed to parse string to integer due to invalid characters!");
        }
    }
}
```

```
/*
Expected Output:
Handled Exception: Failed to parse string to integer due to invalid characters!
*/
```

Problem 5: Multiple Catch Blocks (18.20.5)

```
// Demonstration of multiple catch blocks ordered from specific to general
public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int[] arr = new int[3];
            arr[1] = 50 / 2; // Valid operation
            System.out.println("Array value: " + arr[5]); // Triggers ArrayIndexOutOfBoundsException
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught ArrayIndexOutOfBoundsException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught General Exception: " + e.getMessage());
        }
    }
}
/*
Expected Output:
Caught ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
*/
```

Problem 6: Nested `try-catch` (18.20.6)

```
// Demonstration of placing a try-catch block inside another try block
public class NestedTryCatchDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try block started.");

            try {
                int a = 30 / 0; // Inner exception
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: Handled division by zero.");
            }

            String text = null;
            System.out.println(text.length()); // Outer exception

        } catch (NullPointerException e) {
            System.out.println("Outer catch: Handled null pointer reference.");
        }
    }
}
/*
Expected Output:
Outer try block started.
Inner catch: Handled division by zero.
Outer catch: Handled null pointer reference.
*/
```

Problem 7: `finally` Block Program (18.20.7)

```
// Demonstration of guaranteed finally block execution
public class FinallyDemo {
    public static void main(String[] args) {
        System.out.println("Case 1: No exception thrown");
        testFinally(10, 2);

        System.out.println("\nCase 2: Exception thrown and caught");
        testFinally(10, 0);
    }

    static void testFinally(int a, int b) {
        try {
            int res = a / b;
            System.out.println("Result: " + res);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught in method.");
        } finally {
            System.out.println("Finally block executed ALWAYS!");
        }
    }
}
/*
Expected Output:
Case 1: No exception thrown
Result: 5
Finally block executed ALWAYS!

Case 2: Exception thrown and caught
Exception caught in method.
Finally block executed ALWAYS!
*/
```

Problem 8: `throw` Program (18.20.8)

```
// Demonstration of throwing an exception explicitly using throw keyword
public class ThrowDemo {
    static void validateAge(int age) {
        if (age < 18) {
            // Explicitly throwing an unchecked exception
            throw new ArithmeticException("Access Denied: Candidate must be at least 18 years old to vote.");
        } else {
            System.out.println("Access Granted: Candidate is eligible to vote.");
        }
    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (ArithmeticException e) {
            System.out.println("Caught Explicit Exception: " + e.getMessage());
        }
    }
}
/*
Expected Output:
```

```
Caught Explicit Exception: Access Denied: Candidate must be at least 18 years old to vote.
*/
```

Problem 9: `throws` Program (18.20.9)

```
// Demonstration of declaring exceptions using throws keyword
import java.io.IOException;

public class ThrowsDemo {
    // Declaring that this method might throw an IOException
    static void checkFile() throws IOException {
        throw new IOException("File not found on server disk.");
    }

    public static void main(String[] args) {
        try {
            checkFile(); // Caller must handle or re-throw checked exception
        } catch (IOException e) {
            System.out.println("Handled Checked Exception from method: " + e.getMessage());
        }
    }
}
/*
Expected Output:
Handled Checked Exception from method: File not found on server disk.
*/
```

Problem 10: Custom Exception Program (18.20.10)

```
// Creating a custom unchecked exception by extending RuntimeException
class InvalidBankBalanceException extends RuntimeException {
    public InvalidBankBalanceException(String message) {
        super(message); // Pass custom message to RuntimeException superclass
    }
}

public class CustomExceptionDemo {
    static void withdraw(double balance, double amount) {
        if (amount > balance) {
            throw new InvalidBankBalanceException("Insufficient Funds! Requested: $" + amount + ", Available: $" + balance);
        }
        System.out.println("Withdrawal successful! Remaining balance: $" + (balance - amount));
    }

    public static void main(String[] args) {
        try {
            withdraw(500.0, 750.0);
        } catch (InvalidBankBalanceException e) {
            System.out.println("Custom Exception Caught: " + e.getMessage());
        }
    }
}
/*
Expected Output:
Custom Exception Caught: Insufficient Funds! Requested: $750.0, Available: $500.0
*/
```

Problem 11: Checked Exception Program (18.20.11)

```
// Demonstration of mandatory checked exception handling
import java.io.FileReader;
import java.io.FileNotFoundException;

public class CheckedExceptionDemo {
    public static void main(String[] args) {
        try {
            // FileReader constructor declares FileNotFoundException (Checked Exception)
            FileReader reader = new FileReader("non_existent_file.txt");
        } catch (FileNotFoundException e) {
            System.out.println("Checked Exception Caught: Target file does not exist.");
        }
    }
}
/*
Expected Output:
Checked Exception Caught: Target file does not exist.
*/
```

Problem 12: Unchecked Exception Program (18.20.12)

```
// Demonstration of unchecked exception occurrence and handling
public class UncheckedExceptionDemo {
    public static void main(String[] args) {
        // Unchecked exception (RuntimeException subclass) does not require compiler enforcement
        try {
            String str = "abc";
            char ch = str.charAt(10); // IndexOutOfBoundsException
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Unchecked Exception Caught: " + e.getMessage());
        }
    }
}
/*
Expected Output:
Unchecked Exception Caught: String index out of range: 10
*/
```

Problem 13: `try-with-resources` Program (18.20.13)

```
// Demonstration of automatic resource closure using try-with-resources
import java.io.BufferedReader;
import java.io.StringReader;
import java.io.IOException;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        // Resource initialized inside try(...) statement automatically closes after execution
        try (BufferedReader br = new BufferedReader(new StringReader("Hello, Java Full Stack!"))) {
            String line = br.readLine();
            System.out.println("Read Line: " + line);
        } catch (IOException e) {
            System.out.println("I/O Error: " + e.getMessage());
        }
        // BufferedReader is automatically closed here without requiring an explicit finally block
    }
}
```

```
        System.out.println("Resource automatically closed!");
    }
}
/*
Expected Output:
Read Line: Hello, Java Full Stack!
Resource automatically closed!
*/
```

CHAPTER 1 REVIEW & CHECKLIST

- **Summary:** Exception handling provides a structured model to catch runtime anomalies without abruptly terminating execution.
- **Key Takeaways:** `try` defines dangerous code; `catch` intercepts errors; `finally` cleans resources; `throw` triggers errors explicitly; `throws` declares checked errors.
- **Revision Checklist:** Checked exceptions extend `Exception` directly; Unchecked exceptions extend `RuntimeException`. Subclass `catch` blocks must precede superclass `catch` blocks.

CHAPTER 2 — MULTITHREADING

2.1 Thread Definition

A **Thread** is the smallest lightweight unit of execution within a program (a sequential path of execution).

- **Key Property:** Threads share the same process memory space (Heap and Method area), but each thread maintains its own private Call Stack, Program Counter (PC) register, and local variables.

2.2 Multithreading

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program (threads) to maximize CPU utilization.

- **Goal:** To perform multiple tasks concurrently without blocking the main execution flow or freezing user interfaces.

2.3 Multitasking vs Multithreading

Feature	Multitasking (Process-based)	Multithreading (Thread-based)
Execution	Running multiple independent OS processes simultaneously (e.g., Browser + IDE + Music Player).	Running multiple concurrent paths inside the <i>same</i> process.
Memory	Each process is allocated an isolated memory address space.	All threads share the process's common Heap memory.

Cost	Context switching between processes is heavy and expensive.	Context switching between threads is lightweight and fast.
Isolation	High isolation; process crash does not affect others.	Threads share memory; uncaught exceptions can affect process state.

2.4 `Thread` Class

Java provides the `java.lang.Thread` class to create and manage threads.

- A custom thread class can extend `java.lang.Thread` and override its `run()` method.
- **Limitation:** Since Java does not support multiple class inheritance, extending `Thread` prevents extending any other parent class.

2.5 `Runnable` Interface

The `java.lang.Runnable` functional interface represents a task to be executed by a thread.

- It contains a single abstract method: `public void run()`.
- **Advantage:** Implementing `Runnable` allows the class to extend another class if required, promoting better OOP flexibility and design.

2.6 `run()` Method

The `run()` method contains the job/task code that executes when the thread runs.

- **Signature:** `public void run()`
- Direct calls to `run()` execute the method synchronously on the *current calling thread*, without creating a new thread.

2.7 `start()` Method

The `start()` method registers a new thread with the Thread Scheduler and causes the JVM to call the thread's `run()` method asynchronously on a new call stack.

- **Rule:** Calling `start()` on an already started thread causes an `IllegalThreadStateException`.

2.8 `sleep()` Method

`Thread.sleep(long millis)` pauses execution of the currently running thread for the specified duration (in milliseconds).

- **Characteristics:** `sleep()` is a static method of `Thread`. It does NOT release any held object locks. It throws `InterruptedException` (a checked exception).

2.9 Thread Naming

Every thread in Java has a name for identification during debugging and logging.

- **Default Name:** ``Thread-0``, ``Thread-1``, etc. Main thread name is ``main``.
- **Methods:** ``setName(String name)``, ``getName()``, ``Thread.currentThread().getName()``.

2.10 ``join()`` Method

``join()`` causes the current thread to pause execution and wait until the target thread on which ``join()`` was called completes its execution.

- **Use Case:** Ensuring thread dependencies are satisfied (e.g., Thread B waits for Thread A's data processing to finish before starting).

2.11 Daemon Thread

A **Daemon Thread** is a low-priority background service thread that provides support to user threads (e.g., Garbage Collector, Signal Dispatcher).

- **Key Behavior:** When all user threads finish execution, the JVM automatically terminates and exits, shutting down all remaining daemon threads instantly.
- **Method:** ``setDaemon(boolean on)`` must be called **before** ``start()``; calling it after ``start()`` throws ``IllegalThreadStateException``.

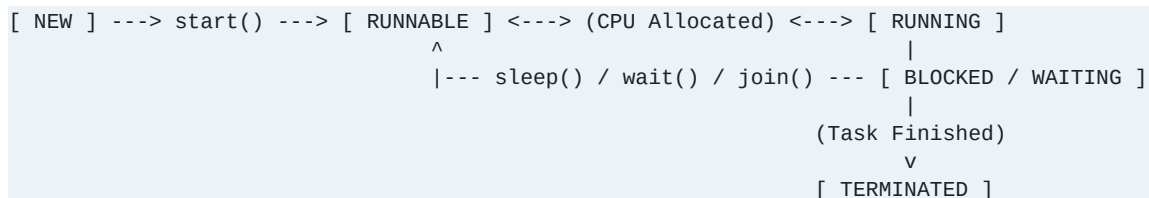
2.12 Thread Priority

Thread priority informs the OS Thread Scheduler about the relative preference of threads.

- **Range:** Integer values from ``1`` (``MIN_PRIORITY``) to ``10`` (``MAX_PRIORITY``). Normal/default priority is ``5`` (``NORM_PRIORITY``).
- **Methods:** ``setPriority(int newPriority)``, ``getPriority()``.
- **Note:** Priority execution is platform/OS dependent and not strictly guaranteed across all OS schedulers.

2.13 Thread Lifecycle (Thread States)

A thread passes through distinct states managed by the JVM and OS scheduler:



1. **NEW:** Thread object created (``new Thread()``), but ``start()`` has not been called.
2. **RUNNABLE:** ``start()`` called; thread is ready to run and waiting in the runnable pool for CPU allocation.
3. **RUNNING:** Thread Scheduler allocated CPU; the code inside ``run()`` is executing.
4. **BLOCKED / WAITING / TIMED_WAITING:** Thread is waiting for a lock (``BLOCKED``), waiting indefinitely (``WAITING`` via ``wait()`` or ``join()``), or waiting for a timer (``TIMED_WAITING`` via ``sleep()`` or ``join(ms)``).

5. **TERMINATED (DEAD):** Execution of `run()` finishes normally or terminates due to an unhandled exception.

2.14 Common Multithreading Programs (Syllabus 19.14.1 to 19.14.8)

Problem 1: Create Thread Using `Thread` Class (19.14.1)

```
// Thread creation by extending Thread class
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Child Thread executing: " + i);
        }
    }
}

public class ThreadClassDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // Thread state: NEW
        t1.start(); // Thread state: RUNNABLE -> RUNNING

        for (int i = 1; i <= 3; i++) {
            System.out.println("Main Thread executing: " + i);
        }
    }
}
/*
Expected Output (Order may vary due to concurrent scheduling):
Main Thread executing: 1
Child Thread executing: 1
Main Thread executing: 2
Child Thread executing: 2
Main Thread executing: 3
Child Thread executing: 3
*/
```

Problem 2: Create Thread Using `Runnable` (19.14.2)

```
// Thread creation by implementing Runnable interface
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable task running on thread: " + Thread.currentThread().getName());
    }
}

public class RunnableDemo {
    public static void main(String[] args) {
        MyRunnable task = new MyRunnable();
        Thread thread = new Thread(task, "WorkerThread"); // Passing task and custom name
        thread.start();
    }
}
/*
```

```
Expected Output:
Runnable task running on thread: WorkerThread
*/
```

Problem 3: `start()` vs `run()` (19.14.3)

```
// Demonstrating difference between start() and run()
class ExecutionModeThread extends Thread {
    @Override
    public void run() {
        System.out.println("Executed by thread: " + Thread.currentThread().getName());
    }
}

public class StartVsRunDemo {
    public static void main(String[] args) {
        ExecutionModeThread t1 = new ExecutionModeThread();

        System.out.println("Direct call to run():");
        t1.run(); // Synchronous call on main thread!

        System.out.println("Call to start():");
        t1.start(); // Asynchronous call on new thread (Thread-0!)
    }
}
/*
Expected Output:
Direct call to run():
Executed by thread: main
Call to start():
Executed by thread: Thread-0
*/
```

Problem 4: `sleep()` Program (19.14.4)

```
// Demonstration of Thread.sleep()
public class SleepDemo {
    public static void main(String[] args) {
        Thread worker = new Thread(() -> {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Processing item " + i);
                try {
                    Thread.sleep(500); // Pause for 500ms
                } catch (InterruptedException e) {
                    System.out.println("Thread interrupted during sleep!");
                }
            }
        });
        worker.start();
    }
}
/*
Expected Output (with 500ms delay between prints):
Processing item 1
Processing item 2
Processing item 3
*/
```

Problem 5: Thread Naming Program (19.14.5)

```
// Demonstration of setting and getting thread names
public class ThreadNamingDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            System.out.println("Running in thread: " + Thread.currentThread().getName());
        });

        System.out.println("Default name: " + t1.getName());
        t1.setName("DataSyncThread");
        System.out.println("New name: " + t1.getName());
        t1.start();
    }
}
/*
Expected Output:
Default name: Thread-0
New name: DataSyncThread
Running in thread: DataSyncThread
*/
```

Problem 6: `join()` Program (19.14.6)

```
// Demonstration of thread join() for sequential coordination
public class JoinDemo {
    public static void main(String[] args) {
        Thread loader = new Thread(() -> {
            System.out.println("Database loading started...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
            System.out.println("Database loading completed!");
        });

        loader.start();

        try {
            System.out.println("Main thread waiting for loader to finish...");
            loader.join(); // Main thread pauses until loader thread finishes!
        } catch (InterruptedException e) {}

        System.out.println("Application startup continued after database load.");
    }
}
/*
Expected Output:
Database loading started...
Main thread waiting for loader to finish...
Database loading completed!
Application startup continued after database load.
*/
```

Problem 7: Daemon Thread Program (19.14.7)

```
// Demonstration of Daemon Thread property
public class DaemonThreadDemo {
```

```
public static void main(String[] args) {
    Thread daemon = new Thread(() -> {
        while (true) {
            System.out.println("Background cleanup running...");
            try {
                Thread.sleep(300);
            } catch (InterruptedException e) {}
        }
    });

    daemon.setDaemon(true); // Mark as daemon thread before start
    daemon.start();

    System.out.println("Main thread running for 1 second...");
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {}
    System.out.println("Main thread finishing. JVM shutting down daemon automatically!");
}
/*
Expected Output:
Main thread running for 1 second...
Background cleanup running...
Background cleanup running...
Background cleanup running...
Main thread finishing. JVM shutting down daemon automatically!
*/
```

Problem 8: Thread Priority Program (19.14.8)

```
// Demonstration of thread priorities
public class PriorityDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> System.out.println("Low Priority Thread running"), "LowPriority");
        Thread t2 = new Thread(() -> System.out.println("High Priority Thread running"), "HighPriority");

        t1.setPriority(Thread.MIN_PRIORITY); // Priority 1
        t2.setPriority(Thread.MAX_PRIORITY); // Priority 10

        System.out.println("t1 Priority: " + t1.getPriority());
        System.out.println("t2 Priority: " + t2.getPriority());

        t1.start();
        t2.start();
    }
}
/*
Expected Output:
t1 Priority: 1
t2 Priority: 10
Low Priority Thread running
High Priority Thread running
*/
```

CHAPTER 2 REVIEW & CHECKLIST

- **Summary:** Multithreading enables lightweight concurrent execution paths within a single Java application.
- **Key Takeaways:** Extend `Thread` or implement `Runnable`; start execution with `start()`, not `run()`; `join()` synchronizes completion; daemon threads automatically exit when all user threads finish.

CHAPTER 3 — SYNCHRONIZATION & INTER-THREAD COMMUNICATION

3.1 Thread Safety & Data Races

When multiple threads attempt to read and write to a **shared resource** (e.g., a shared instance variable) concurrently without proper control, a **Data Race / Race Condition** occurs.

- **Race Condition:** The final state of the shared resource depends on the arbitrary non-deterministic thread interleaving order.
- **Thread Safety:** Ensuring that a shared resource behaves correctly and predictably regardless of how many threads access it concurrently.

3.2 Object Lock & Monitor Concept

Every object in Java has an internal implicit lock called a **Monitor / Intrinsic Lock / Mutex**.

- When a thread enters a `synchronized` method or block on an object, it automatically acquires that object's intrinsic lock.
- **Rule:** Only **one thread** can hold an object's intrinsic lock at any given time. Other threads attempting to acquire the same lock are moved to the `BLOCKED` state until the lock owner releases it upon exiting the synchronized code.

3.3 Synchronized Method

Declaring a method with the `synchronized` modifier locks the entire method.

- **Instance Synchronized Method:** Acquires the lock on `this` instance object.
- **Static Synchronized Method:** Acquires the lock on the `Class` object (`ClassName.class`).

3.4 Synchronized Block

A `synchronized` block locks only a specific critical section of code rather than an entire method, improving performance by reducing lock scope.

- **Syntax:** `synchronized (lockObject) { // Critical Section }`

3.5 Shared Resource

A memory location, field, or object accessible by multiple threads concurrently (e.g., bank account balance, inventory count, database connection pool).

3.6 Deadlock

A **Deadlock** is a situation where two or more threads are blocked forever, waiting for locks held by each other in a circular dependency chain.

- **Deadlock Conditions:**

1. Thread 1 holds Lock A and requests Lock B.
2. Thread 2 holds Lock B and requests Lock A.
3. Neither thread releases its lock, resulting in an indefinite freeze.

3.7 Inter-Thread Communication

Inter-Thread Communication allows synchronized threads to communicate with each other using notification mechanisms provided by the `java.lang.Object` class:

- `wait()`: Causes the current thread to release its held object lock and wait until another thread calls `notify()` or `notifyAll()` on the same lock object.
- `notify()`: Wakes up a single thread that is waiting on the object's lock.
- `notifyAll()`: Wakes up all threads waiting on the object's lock.
- **Mandatory Rule:** `wait()`, `notify()`, and `notifyAll()` MUST be called from within a `synchronized` context (method or block), otherwise an `IllegalMonitorStateException` is thrown.

3.8 Producer–Consumer Problem

A classic multithreading synchronization pattern where one thread (Producer) generates data and places it into a shared buffer, while another thread (Consumer) removes and processes data from the buffer.

- **Coordination:** `wait()` pauses the producer if the buffer is full; `wait()` pauses the consumer if the buffer is empty. `notify()` signals when buffer status changes.

3.9 Synchronization Coding Problems (Syllabus 20.12.1 to 20.12.6)

Problem 1: Synchronized Method Program (20.12.1)

```
// Demonstration of thread-safe counter using a synchronized method
class BankAccount {
    private double balance = 1000.0;

    // Synchronized method locks the entire instance method
    public synchronized void withdraw(String name, double amount) {
        if (balance >= amount) {
            System.out.println(name + " is withdrawing $" + amount);
        }
    }
}
```

```
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {}
        balance -= amount;
        System.out.println(name + " completed withdrawal. Remaining Balance: $" + balance);
    } else {
        System.out.println(name + " failed withdrawal! Insufficient Balance: $" + balance);
    }
}
}

public class SynchronizedMethodDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread user1 = new Thread(() -> account.withdraw("User-1", 700.0));
        Thread user2 = new Thread(() -> account.withdraw("User-2", 500.0));

        user1.start();
        user2.start();
    }
}
/*
Expected Output (Guaranteed sequential thread safety):
User-1 is withdrawing $700.0
User-1 completed withdrawal. Remaining Balance: $300.0
User-2 failed withdrawal! Insufficient Balance: $300.0
*/
```

Problem 2: Synchronized Block Program (20.12.2)

```
// Demonstration of synchronized block locking a critical section
class Printer {
    public void printDocument(String docName) {
        // Non-critical code before lock
        System.out.println("Preparing document: " + docName + " on thread: " + Thread.currentThread().getName());

        // Critical section locked on this Printer instance
        synchronized (this) {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Printing " + docName + " - Page " + i);
            }
        }
    }
}

public class SynchronizedBlockDemo {
    public static void main(String[] args) {
        Printer printer = new Printer();

        Thread t1 = new Thread(() -> printer.printDocument("Report.pdf"), "Thread-1");
        Thread t2 = new Thread(() -> printer.printDocument("Invoice.pdf"), "Thread-2");

        t1.start();
        t2.start();
    }
}
/*
```

```
Expected Output (Pages for each document are kept together continuously):
Preparing document: Report.pdf on thread: Thread-1
Preparing document: Invoice.pdf on thread: Thread-2
Printing Report.pdf - Page 1
Printing Report.pdf - Page 2
Printing Report.pdf - Page 3
Printing Invoice.pdf - Page 1
Printing Invoice.pdf - Page 2
Printing Invoice.pdf - Page 3
*/
```

Problem 3: Shared Counter Program (20.12.3)

```
// Demonstration of atomicity in shared counter using synchronized block
class SharedCounter {
    private int count = 0;

    public void increment() {
        synchronized (this) {
            count++; // Atomic increment
        }
    }

    public int getCount() {
        return count;
    }
}

public class SharedCounterDemo {
    public static void main(String[] args) throws InterruptedException {
        SharedCounter counter = new SharedCounter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Counter Value (Expected 2000): " + counter.getCount());
    }
}
/*
Expected Output:
Final Counter Value (Expected 2000): 2000
*/
```

Problem 4: Deadlock Demonstration (20.12.4)

```
// Demonstration of creating and recognizing a Deadlock scenario
public class DeadlockDemo {
    private static final Object LockA = new Object();
    private static final Object LockB = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (LockA) {
                System.out.println("Thread 1: Holding Lock A...");
                try { Thread.sleep(100); } catch (InterruptedException e) {}
                System.out.println("Thread 1: Waiting for Lock B...");
                synchronized (LockB) {
                    System.out.println("Thread 1: Acquired Lock B!");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (LockB) {
                System.out.println("Thread 2: Holding Lock B...");
                try { Thread.sleep(100); } catch (InterruptedException e) {}
                System.out.println("Thread 2: Waiting for Lock A...");
                synchronized (LockA) {
                    System.out.println("Thread 2: Acquired Lock A!");
                }
            }
        });

        t1.start();
        t2.start();
    }
}
/*
Expected Output (Program freezes due to circular lock dependency):
Thread 1: Holding Lock A...
Thread 2: Holding Lock B...
Thread 1: Waiting for Lock B...
Thread 2: Waiting for Lock A...
(Execution freezes indefinitely)
*/
```

Problem 5: `wait()` / `notify()` Program (20.12.5)

```
// Demonstration of wait() and notify() inter-thread communication
class MessageQueue {
    private String message;
    private boolean empty = true;

    public synchronized String read() {
        while (empty) {
            try {
                wait(); // Wait until message is published
            } catch (InterruptedException e) {}
        }
        empty = true;
        notify(); // Signal writer thread
        return message;
    }

    public synchronized void write(String msg) {
```

```
        while (!empty) {
            try {
                wait(); // Wait until message is read
            } catch (InterruptedException e) {}
        }
        empty = false;
        this.message = msg;
        System.out.println("Published: " + msg);
        notify(); // Signal reader thread
    }
}

public class WaitNotifyDemo {
    public static void main(String[] args) {
        MessageQueue queue = new MessageQueue();

        Thread writer = new Thread(() -> {
            queue.write("Data Package 1");
        });

        Thread reader = new Thread(() -> {
            String msg = queue.read();
            System.out.println("Consumed: " + msg);
        });

        reader.start();
        writer.start();
    }
}
/*
Expected Output:
Published: Data Package 1
Consumed: Data Package 1
*/
```

Problem 6: Producer–Consumer Program (20.12.6)

```
// Demonstration of Producer-Consumer problem using wait() and notify()
import java.util.LinkedList;

class SharedBuffer {
    private final LinkedList<Integer> list = new LinkedList<>();
    private final int capacity = 2;

    public synchronized void produce(int value) throws InterruptedException {
        while (list.size() == capacity) {
            System.out.println("Buffer FULL. Producer waiting...");
            wait();
        }
        list.add(value);
        System.out.println("Producer produced: " + value);
        notifyAll();
    }

    public synchronized void consume() throws InterruptedException {
        while (list.isEmpty()) {
            System.out.println("Buffer EMPTY. Consumer waiting...");
            wait();
        }
    }
}
```

```
        int val = list.removeFirst();
        System.out.println("Consumer consumed: " + val);
        notifyAll();
    }
}

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        SharedBuffer buffer = new SharedBuffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    buffer.produce(i * 10);
                    Thread.sleep(200);
                }
            } catch (InterruptedException e) {}
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) {
                    buffer.consume();
                    Thread.sleep(400);
                }
            } catch (InterruptedException e) {}
        });

        producer.start();
        consumer.start();
    }
}
/*
Expected Output:
Producer produced: 10
Consumer consumed: 10
Producer produced: 20
Consumer consumed: 20
Producer produced: 30
Consumer consumed: 30
*/
```

CHAPTER 3 REVIEW & CHECKLIST

- **Summary:** Synchronization locks critical sections to ensure thread safety; `wait()` and `notify()` coordinate inter-thread execution order.
- **Key Takeaways:** Synchronized block > Synchronized method (finer control); lock must be held to call `wait()` / `notify()`; avoid deadlocks by ordering lock acquisitions consistently.

BOOK 4 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 4 REVISION SHEET

- **Exception Hierarchy:** `Throwable` is root; `Error` is unrecoverable system issue; `Exception` is application recoverable issue.
 - **Checked vs Unchecked Exceptions:** Checked exceptions inherit directly from `Exception` and require compile-time handling (`try-catch` or `throws`); Unchecked exceptions inherit from `RuntimeException` and occur at runtime.
 - **`try-catch-finally` Mechanics:** `finally` ALWAYS executes (except `System.exit(0)`). Subclass `catch` blocks MUST be placed before superclass `catch` blocks.
 - **`throw` vs `throws`:** `throw` explicitly raises an exception object inside code; `throws` declares potential checked exceptions in a method signature.
 - **Multithreading:** Lightweight execution path sharing Heap/Method area; separate Stack/PC register per thread.
 - **Creation Methods:** Extend `Thread` class OR implement `Runnable` interface. Always call `start()` to create a new call stack.
 - **Thread States:** NEW $\rightarrow$ RUNNABLE $\rightarrow$ RUNNING $\rightarrow$ BLOCKED/WAITING $\rightarrow$ TERMINATED.
 - **Synchronization:** Intrinsic object locks (monitors) enforce mutual exclusion to prevent data races on shared resources.
 - **Inter-Thread Communication:** `wait()`, `notify()`, and `notifyAll()` methods belong to `java.lang.Object` and MUST be called inside a `synchronized` block/method.
-

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

Question 1: What is the difference between `final`, `finally`, and `finalize()`?

Answer:

- **`final`:** A keyword/modifier used to make variables constant (immutable value), methods unoverridable, and classes uninheritable.
- **`finally`:** A block used in exception handling that ALWAYS executes after `try-catch` blocks to perform mandatory resource cleanup.
- **`finalize()`:** A method in `java.lang.Object` invoked by the Garbage Collector before an object is reclaimed (deprecated since Java 9).

Question 2: Why are `wait()`, `notify()`, and `notifyAll()` defined in `Object` class instead of `Thread` class?

Answer:

`wait()`, `notify()`, and `notifyAll()` operate on the **intrinsic lock (monitor)** of an object, not on the thread itself. Since every object in Java has a monitor lock and can act as a shared resource, these methods are defined on `java.lang.Object`.

Question 3: What happens if you call `start()` twice on the same thread instance?

Answer:

Calling `start()` a second time on an already started thread throws an `IllegalThreadStateException` at runtime because the thread has already transitioned out of the `NEW` state.

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: Return Value with `finally`

```
public class FinallyReturnDemo {
    public static int test() {
        try {
            return 10;
        } catch (Exception e) {
            return 20;
        } finally {
            return 30; // finally return overrides previous returns!
        }
    }
    public static void main(String[] args) {
        System.out.println("Returned: " + test());
    }
}
```

- **Prediction:** `Returned: 30`
- **Explanation:** The `finally` block always executes before returning. Returning a value inside `finally` overrides any return statement executed in `try` or `catch`.

Question 2: Catch Block Order

```
public class CatchOrderDemo {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught");
        } catch (Exception e) {
            System.out.println("Exception caught");
        }
    }
}
```

- **Prediction:** `ArithmeticException caught`
- **Explanation:** The exact matching specific subclass `ArithmeticException` catches the exception before reaching the general `Exception` catch block.

4. DEBUGGING PRACTICE & COMMON MISTAKES

Trap 1: Missing Base Case Causing `StackOverflowError`

```
// Buggy Code
class RecursionBug {
    static void printNum(int n) {
        System.out.println(n);
        printNum(n + 1); // Missing termination condition!
    }
}
// Fix: Add base case `if (n > 100) return;`
```

Trap 2: Calling `wait()` Outside `synchronized` Context

```
// Buggy Code
public class WaitBug {
    public static void main(String[] args) throws InterruptedException {
        Object lock = new Object();
        lock.wait(); // Throws IllegalMonitorStateException!
    }
}
// Fix: Wrap inside `synchronized (lock) { lock.wait(); }`
```

5. CONCEPT COMPARISON QUESTIONS

Topic	Option A	Option B	Option C
Exception Types	Checked Exception (`IOException`)	Unchecked Exception (`NullPointerException`)	Error (`StackOverflowError`)
Thread Creation	`extends Thread`	`implements Runnable`	Lambda (`() -> {}`)
Lock Scope	Synchronized Method	Synchronized Block	ReentrantLock
Communication	`wait()` (Releases lock, waits)	`notify()` (Wakes 1 thread)	`notifyAll()` (Wakes all threads)

6. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: How does Java handle runtime exceptions vs checked exceptions during compilation?

Candidate: Checked exceptions inherit directly from `java.lang.Exception` (excluding `RuntimeException`). The compiler strictly enforces handling checked exceptions via `try-catch` or declaring them in the signature using `throws`. Unchecked exceptions inherit from `java.lang.RuntimeException` and represent logic flaws like `NullPointerException` or `ArithmeticException` — the compiler does not enforce handling them at compile time.

Interviewer: Excellent! What is the difference between `start()` and `run()` in multithreading?

Candidate: Direct invocation of `run()` executes the method synchronously on the current thread without spawning a new thread. Calling `start()` registers a new thread with the JVM Thread Scheduler, creates a new call stack, and asynchronously invokes `run()` on the new thread.

7. COMPLETE BOOK 4 SYLLABUS COVERAGE CHECKLIST

- [x] **Section 18: Exception Handling (18.1 to 18.20.13)**

- All concepts: Exception definition, propagation, hierarchy, `try-catch-finally`, `throw`, `throws`, checked/unchecked exceptions, custom exceptions, overriding rules, `try-with-resources`.

- All 13 coding problems solved (18.20.1 to 18.20.13).

- [x] **Section 19: Multithreading (19.1 to 19.14.8)**

- All concepts: `Thread` class, `Runnable` interface, `run()`, `start()`, `sleep()`, naming, `join()`, daemon threads, priority, thread lifecycle states.

- All 8 coding problems solved (19.14.1 to 19.14.8).

- [x] **Section 20: Synchronization & Inter-Thread Communication (20.1 to 20.12.6)**

- All concepts: Thread safety, object locks/monitors, `synchronized` method/block, shared resources, deadlock, `wait()`, `notify()`, producer-consumer pattern.

- All 6 coding problems solved (20.12.1 to 20.12.6).

BOOK 4 COMPLETE — 100% MASTER SYLLABUS COVERAGE VERIFIED.